

Resolución del Problema de N-Reinas mediante el Algoritmo de Grover

Ignacio Andres Schwindt (03229/0) · Viernes 17 de Junio de 2026

RESUMEN

Se presenta una implementación del algoritmo de búsqueda cuántica de Grover aplicada al problema combinatorio de las N-reinas. La codificación asigna $k = \lceil \log_2 N \rceil$ qubits por reina para representar su columna, eliminando conflictos de fila por construcción. El oráculo detecta conflictos de columna y diagonal mediante enumeración explícita de patrones y qubits flag, aplicando fase -1 a las configuraciones válidas a través del esquema *compute-phase-uncompute*. Se verifica la corrección completa para $N = 4$ (14 qubits, 8 iteraciones, $P(\text{éxito}) \approx 99.6\%$) y se valida el oráculo de forma aislada para $N = 5$ (30 qubits), confirmando que las 10 soluciones conocidas son correctamente identificadas. Se discuten las limitaciones de escalabilidad y posibles optimizaciones mediante sumadores cuánticos.

PALABRAS CLAVE Computación cuántica · Algoritmo de Grover · N-reinas · Oráculo cuántico · Qiskit · Búsqueda no estructurada

1. INTRODUCCIÓN

El problema de las N-reinas consiste en ubicar N reinas en un tablero de $N \times N$ de modo que ninguna amenace a otra: sin compartir fila, columna ni diagonal. Formulad originalmente por Max Bezzel en 1848 y estudiado extensivamente por Gauss, el problema se ha convertido en un clásico de la teoría combinatoria y la ciencia de la computación [4]. Para $N = 4$ existen exactamente 2 soluciones; para $N = 5$, hay 10; para $N = 8$ (tablero estándar), 92; y la cantidad crece rápidamente. La búsqueda clásica por fuerza bruta tiene complejidad $\mathcal{O}(N!)$ al explorar permutaciones de columnas, y aunque algoritmos de backtracking mejoran esto en la práctica, la complejidad en el peor caso sigue siendo exponencial.

El **algoritmo de Grover** [1], publicado en 1996, ofrece una aceleración cuadrática para problemas de búsqueda no estructurada: si hay M soluciones en un espacio de tamaño $\mathcal{N}$, las encuentra en $\mathcal{O}(\sqrt{\mathcal{N}/M})$ consultas al oráculo, contra $\mathcal{O}(\mathcal{N}/M)$ en el caso clásico. Esta aceleración es óptima: Bennett, Bernstein, Brassard y Vazirani demostraron que ningún algoritmo cuántico puede hacerlo mejor para búsqueda no estructurada [5].

En este trabajo se presenta una implementación en Python utilizando Qiskit [3], simulada con Aer, que cubre la codificación cuántica del tablero, el diseño del oráculo, el difusor de Grover, y resultados para $N = 4$ (ejecución completa) y $N = 5$ (val. oráculo).

Adicionalmente, se discute el impacto del mapeo de las restricciones combinatorias hacia compuertas reversibles. Aunque la ejecución física de algoritmos con tal profundidad aún está limitada por el ruido inherente a los procesadores NISQ (*Noisy Intermediate-Scale Quantum*), esta exploración establece un caso de estudio útil sobre cómo estructurar y validar oráculos de decisión complejos de forma modular.

2. EL ALGORITMO DE GROVER

2.1 Principio y flujo general

Grover opera sobre n qubits con $\mathcal{N} = 2^n$ estados. El oráculo U_f aplica $U_f |x\rangle = (-1)^{f(x)} |x\rangle$ (*phase kickback* a las soluciones). El algoritmo (Fig. 1) tiene tres etapas: (1) Inicialización en superposición uniforme $|\psi_0\rangle = H^{\otimes n} |0\rangle^{\otimes n}$. (2) Iteración de Grover (repetida t^* veces): oráculo U_f + difusor $D = 2|\psi_0\rangle\langle\psi_0| - I$. (3) Medición en la base computacional.

La medición colapsa el estado del sistema, revelando la solución con una alta probabilidad si se escogió el número correcto de iteraciones. Este comportamiento probabilístico es propio de los algoritmos cuánticos, donde la interferencia constructiva amplifica la amplitud de los estados que cumplen con las condiciones del oráculo.

Algoritmo de Grover para N-Reinas

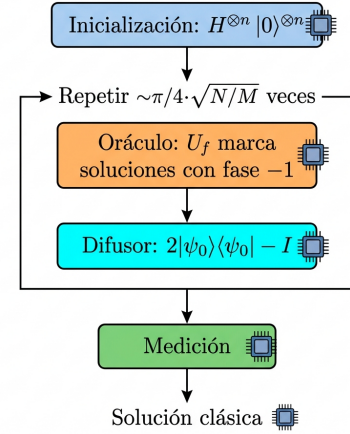


Figura 1. Flujo del algoritmo de Grover para N-reinas.

2.2 Interpretación geométrica

El estado del sistema vive en un plano bidimensional generado por:

$$|\alpha\rangle = \frac{1}{\sqrt{\mathcal{N} - M}} \sum_{f(x)=0} |x\rangle, \quad |\beta\rangle = \frac{1}{\sqrt{M}} \sum_{f(x)=1} |x\rangle \quad (1)$$

El estado inicial $|\psi_0\rangle$ forma un ángulo $\theta/2 = \arcsin \sqrt{M/\mathcal{N}}$ con $|\alpha\rangle$. Cada iteración rota el estado un ángulo θ hacia $|\beta\rangle$.

2.3 Iteraciones óptimas

El número óptimo de iteraciones es:

$$t^* = \left\lfloor \frac{\pi}{2\theta} - \frac{1}{2} \right\rfloor, \quad \theta = 2 \arcsin \sqrt{\frac{M}{\mathcal{N}}} \quad (2)$$

Para $N = 4$: $\mathcal{N} = 256$, $M = 2$, $\theta \approx 0.177$ rad, $t^* = 8$, $P(\text{éxito}) \approx 99.6\%$.

Nota: Si se excede t^* , la probabilidad *disminuye* — el algoritmo “pasa de largo” la solución. Este fenómeno no tiene análogo clásico.

3. CODIFICACIÓN CUÁNTICA

3.1 Eliminación de la simetría de fila

La primera decisión de diseño es fijar implícitamente cada reina i en la fila i . Esto elimina conflictos de fila por construcción y reduce el espacio de búsqueda de N^{2N} (posiciones arbitrarias) a N^N (solo columnas por reina).

3.2 Registro de estado

La columna de la reina i se codifica en $k = \lceil \log_2 N \rceil$ qubits binarios. Los qubits $q_{ik}, q_{ik+1}, \dots, q_{ik+k-1}$ representan la columna en base 2, con q_{ik} como bit menos significativo (LSB). Para $N=4$: $k=2$, registro total de $n = Nk = 8$ qubits (Fig. 2).

Codificación Cuántica de N-Reinas, N=4

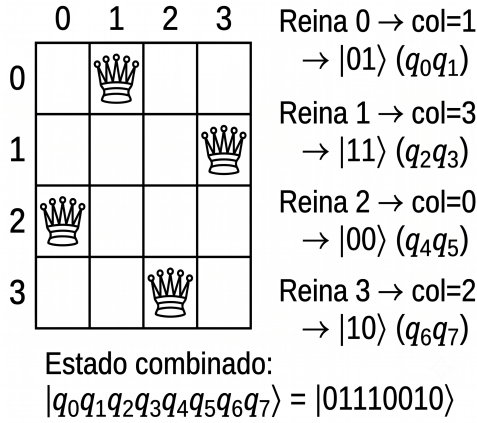


Figura 2. Codificación de $[1, 3, 0, 2]$ para $N = 4$: 2 qubits por reina.

3.3 Qubits ancilla (flags)

El oráculo requiere qubits auxiliares: **Flags de pares**: un qubit por cada par (i, j) con $i < j$, total $\binom{N}{2}$ flags. **Flags de rango** (solo cuando $2^k > N$): un qubit por reina para detectar columnas fuera de $[0, N)$. Para $N = 4$, $2^2 = 4 = N$, no se necesitan; para $N = 5$ ($2^3 = 8 > 5$), se agregan 5 flags.

Cuadro 1. Recursos del circuito según N .

N	k	Estado	F. pares	F. rango	Total	Sols.
4	2	8	6	0	14	2
5	3	15	10	5	30	10
6	3	18	15	6	39	4

4. DISEÑO DEL ORÁCULO

El oráculo es el componente central del circuito. Aplica fase -1 exclusivamente a los estados que representan configuraciones válidas, en tres etapas (Fig. 3): *compute*, *phase flip* y *uncompute*.

Estructura del Oráculo de Grover para N-Reinas

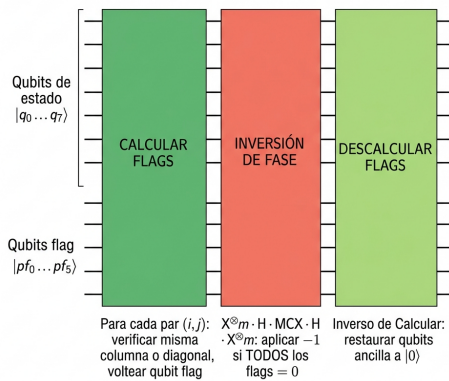


Figura 3. Estructura: Compute → Phase Flip → Uncompute.

4.1 Compute Flags

Para cada par (i, j) , $i < j$, se verifica conflicto ($c_i = c_j$ o $|c_i - c_j| = j - i$) mediante **enumeración explícita de patrones** (Fig. 4).

Costo Computacional: Esta verificación minimiza el ancho aplicando compuertas X *in-place*, pero su profundidad escala como $\mathcal{O}(N^3 \log^2 N)$ debido a las Toffoli generalizadas (MCX) requeridas. Esto explica el enorme costo en $N \geq 5$.

Detección de conflictos en el Oráculo de N-Reinas

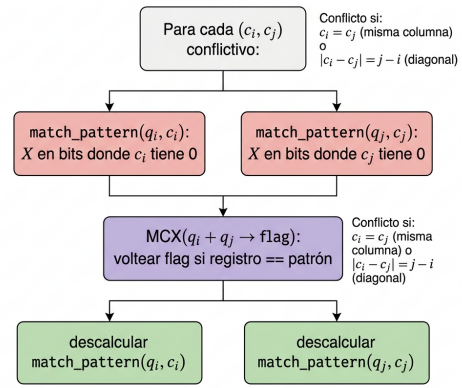


Figura 4. Detección de conflicto (c_i, c_j) con *match_pattern*.

4.1.1 Función *match_pattern*

Esta es la pieza clave de la detección. Dado un registro de k qubits y un valor objetivo v , aplica puertas X a cada qubit cuyo bit correspondiente en v es 0:

$$\bigotimes_{b=0}^{k-1} X^{1-v_b} |v\rangle = |1\rangle^{\otimes k} \quad (3)$$

Esto transforma la condición “el registro vale v ” en “todos los qubits valen 1”, que es exactamente lo que detecta una puerta MCX (Toffoli generalizada). Un MCX posterior voltear el flag correspondiente. Finalmente, se aplica la misma secuencia de X para restaurar el registro ($X^2 = I$). Para un estado computacional fijo $|c_i, c_j\rangle$, exactamente una combinación de patrones coincide, garantizando que el flag se voltear 0 ó 1 veces.

4.1.2 Detección de fuera de rango

Para N que no es potencia de 2 (e.g. $N = 5$ con $k = 3$), los valores de columna $\{N, N+1, \dots, 2^k-1\}$ son inválidos. Un flag adicional por reina se activa si su columna codifica un valor $\geq N$, usando la misma técnica de *match_pattern*.

4.2 Phase Flip

Una vez computados todos los flags, se aplica -1 condicionada a que todos valgan 0 (sin conflictos ni columnas fuera de rango):

$$X^{\otimes m} \cdot (H \cdot MCX \cdot H) \cdot X^{\otimes m} \quad (4)$$

donde m es el número total de flags. La secuencia $X^{\otimes m}$ convierte $|0\rangle^{\otimes m}$ en $|1\rangle^{\otimes m}$ (que dispara el MCX). El truco $H \cdot MCX \cdot H$ implementa un MCZ (fase controlada):

$$H X H = Z \implies H \cdot MCX(\vec{c} \rightarrow t) \cdot H = MCZ(\vec{c}, t) \quad (5)$$

4.3 Uncompute Flags

Para que el oráculo sea unitario y reutilizable entre iteraciones, las ancillas deben volver a $|0\rangle$. Se deshace la etapa de Compute aplicando las mismas operaciones en orden inverso. Dado que X y MCX son autoinversas, esta inversión es directa.

COMPARACIÓN DE CODIFICACIÓN

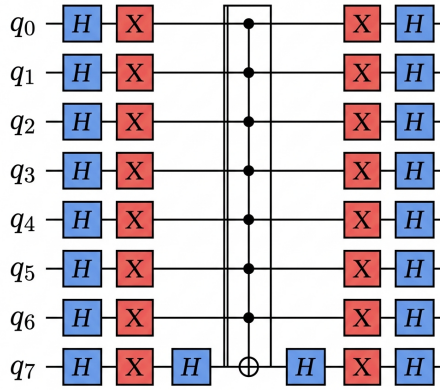
A diferencia de formulaciones basadas en simuladores cuánticos de propósito especial con átomos en redes ópticas [8], y de mapeos espaciales que utilizan N^2 qubits asignando uno a cada casillero [6, 7], el enfoque de codificación por columna presentado aquí requiere solo $N \lceil \log_2 N \rceil$ qubits de estado. Aunque esto reduce drásticamente el espacio de Hilbert necesario y garantiza por construcción que no haya conflictos de fila, conlleva el costo de un oráculo de decisión con mayor profundidad para resolver el resto de las restricciones combinatorias.

5. EL DIFUSOR DE GROVER

Implementa la reflexión $D = 2|\psi_0\rangle\langle\psi_0| - I$ (inversión sobre la media) sobre los n qubits de estado. Su descomposición en puertas es:

$$D = H^{\otimes n} \cdot X^{\otimes n} \cdot MCZ \cdot X^{\otimes n} \cdot H^{\otimes n} \quad (6)$$

El circuito (Fig. 5) opera: (1) pasar a la base de Hadamard con $H^{\otimes n}$; (2) marcar $|0 \dots 0\rangle$ con $X^{\otimes n}$ seguido de MCZ; (3) deshacer $X^{\otimes n}$ y volver a la base computacional.



$$\text{Difusor} = 2|\psi_0\rangle\langle\psi_0| - I$$

Figura 5. Circuito del difusor para 8 qubits de estado.

Efecto sobre las amplitudes. Sea $\bar{a}$ la amplitud media. El difusor transforma cada amplitud $a_x \rightarrow 2\bar{a} - a_x$: los estados con amplitud por debajo de la media se atenúan, mientras que las soluciones (con amplitud negativa grande) se reflejan a valores positivos grandes.

6. RESULTADOS

6.1 $N = 4$: ejecución completa

Para $N = 4$, el circuito completo se simuló con el backend AerSimulator (método statevector) durante 4096 shots.

Cuadro 2. Parámetros del circuito para $N = 4$.

Parámetro	Valor
Qubits de estado	8 (2 bits/columna)
Flags de pares	6
Qubits totales	14
Espacio de búsqueda	$2^8 = 256$
Soluciones clásicas	2
Iteraciones óptimas	8
Profundidad (transpilada)	$\sim 41\,000$

Las dos soluciones válidas, $[1, 3, 0, 2]$ y $[2, 0, 3, 1]$, concentran $\sim 99.6\%$ de la probabilidad total ($\sim 49.6\%$ cada una, Fig. 6). La decodificación del bitstring medido se realiza invirtiendo el orden de bits (Qiskit imprime MSB primero) y reconstruyendo la columna de cada reina: $\text{col}_i = \sum_{b=0}^{k-1} c_{ik+b} \cdot 2^b$.

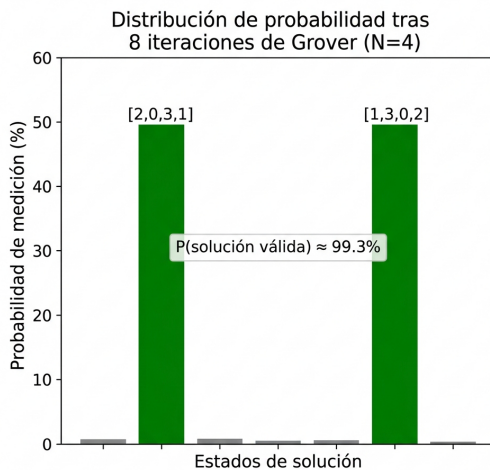


Figura 6. Probabilidad tras 8 iteraciones ($N = 4$). Las dos soluciones dominan con $\sim 49.6\%$ cada una; ruido $< 1\%$.

6.2 $N = 5$: validación del oráculo

El circuito completo para $N = 5$ requiere 30 qubits y ~ 44 iteraciones ($\sim 6 \times 10^5$ compuertas), fuera del alcance práctico de la simulación clásica. Sin embargo, se validó el oráculo de forma aislada, preparando estados computacionales conocidos y midiendo los flags:

Las **10 soluciones** clásicas (obtenidas por fuerza bruta con `classical_solutions`) resultaron en 0 flags activos en todos los casos. Los casos negativos testeados incluyeron:

- $[0, 0, 0, 0, 0]$: todas en misma columna.
- $[0, 1, 2, 3, 4]$: diagonal principal.
- $[0, 2, 4, 1, 7]$: columna 7 fuera de rango (≥ 5).
- $[1, 3, 5, 0, 2]$: columna 5 fuera de rango.

Todos fueron correctamente rechazados con flags > 0 . Esto confirma que el oráculo separa perfectamente las configuraciones válidas del resto del espacio de búsqueda.

7. COMPLEJIDAD Y LIMITACIONES

7.1 Complejidad del oráculo

La enumeración explícita de patrones produce: $\binom{N}{2} = \mathcal{O}(N^2)$ pares de reinas; hasta $\mathcal{O}(N)$ combinaciones conflictivas por par; cada verificación: $\mathcal{O}(k)$ puertas X más un MCX de $2k$ controles. Total: $\mathcal{O}(N^3)$ llamadas a MCX y $\mathcal{O}(N^3 \log N)$ compuertas elementales. Una implementación con **sumadores cuánticos** (*quantum adders*) permitiría comparar columnas aritméticamente, reduciendo los MCX por par de $\mathcal{O}(N)$ a $\mathcal{O}(\text{poly}(k))$.

7.2 Escalabilidad

Cuadro 3. Viabilidad de simulación según N .

N	Qubits	Iters.	Simulable
4	14	8	Sí (statevector)
5	30	44	Solo oráculo (MPS)
6	39	~ 200	No (simulador clásico)

Para $N \geq 5$, las dos vías hacia la ejecución end-to-end son: (1) hardware cuántico real con fidelidad y T_1/T_2 suficientes (aspiracional en 2026); o (2) reescritura del oráculo con sumadores cuánticos para reducir la profundidad.

8. CONCLUSIONES

1. **Corrección verificada.** Para $N = 4$, el algoritmo completo encuentra las 2 soluciones con $P \approx 99.6\%$. Para $N = 5$, el oráculo clasifica correctamente las 10 soluciones y rechaza todos los casos inválidos testeados.
2. **Aceleración cuadrática.** El algoritmo consulta el oráculo $\mathcal{O}(\sqrt{N/M})$ veces, comparado con $\mathcal{O}(N/M)$ en búsqueda clásica. Para $N = 4$: 8 iteraciones vs. ~ 128 consultas clásicas.
3. **Diseño modular.** La separación en codificación, oráculo (compute-phase-uncompute) y difusor permite extender o reemplazar componentes de forma independiente.
4. **Limitación principal.** La enumeración explícita genera circuitos profundos ($\sim 41\,000$ de profundidad para $N = 4$). Sumadores cuánticos reducirían significativamente la profundidad para $N \geq 5$.
5. **Perspectiva.** La ejecución en hardware cuántico real requeriría tiempos de coherencia (T_1, T_2) muy superiores a los disponibles actualmente, además de corrección de errores cuánticos. No obstante, la implementación sirve como demostración conceptual de la ventaja cuadrática de Grover sobre problemas combinatorios.

REFERENCIAS

- [1] L. K. Grover, "A fast quantum mechanical algorithm for database search," *Proc. 28th ACM STOC*, pp. 212–219, 1996.
- [2] M. A. Nielsen and I. L. Chuang, *Quantum Computation and Quantum Information*, Cambridge Univ. Press, 10th Ann. Ed., 2010.
- [3] Qiskit Development Team, "Qiskit: An open-source framework for quantum computing," 2024.
- [4] J. Bell and B. Stevens, "A survey of known results and research areas for n -queens," *Discrete Math.*, vol. 309, no. 1, pp. 1–31, 2009.
- [5] C. H. Bennett, E. Bernstein, G. Brassard, and U. Vazirani, "Strengths and weaknesses of quantum computing," *SIAM J. Comput.*, vol. 26, no. 5, pp. 1510–1523, 1997.
- [6] R. Jha, D. Das, A. Dash, S. Jayaraman, B. K. Behera, and P. K. Panigrahi, "A novel quantum N -queens solver algorithm and its simulation and application to satellite communication using IBM Quantum Experience," arXiv:1806.10221 [quant-ph], 2018.
- [7] S. G. S. Santhosh, P. Joshi, A. Barui, and P. K. Panigrahi, "A quantum approach to solve N -queens problem," arXiv:2312.16312 [quant-ph], 2023.
- [8] V. Torggler, P. Aumann, H. Ritsch, and W. Lechner, "A quantum N -queens solver," *Quantum*, vol. 3, p. 149, 2019. arXiv:1803.00735 [quant-ph].